

The Utility Problem

Distribution network models are not static. They evolve continuously as utilities commission new assets, retire aging infrastructure, reconfigure feeders for load balancing, and respond to fault events. Each of these activities produces a new version of the network model — a snapshot of the grid at a specific point in time. Over months and years, a utility accumulates many such snapshots across its GIS, OMS, and planning systems, each potentially representing a different state of the same feeder or circuit.

Beyond tracking historical changes, utilities increasingly need to compare models for scenario planning purposes. Engineers routinely ask questions such as: What does the feeder look like after the proposed substation upgrade is complete? How does the planned reconfiguration differ from today's topology? What changed between the model used in last year's capacity study and the current operational model? Answering these questions currently requires engineers to compare models manually — a slow, labor-intensive process that is difficult to audit and prone to human error.

Without an automated comparison tool, discrepancies between model versions can propagate silently into planning studies, asset management decisions, and operational switching procedures — producing incorrect results that are only discovered after costly downstream consequences. The need for a systematic, feeder-level diff capability is therefore both an operational data governance requirement and a strategic planning enabler.

The Data Solutions and Methods

The Awesense Platform ingests GIS network model data with full timestamp preservation. Every import of a GIS dataset is stored as a versioned snapshot rather than overwriting the previous state. As a result, the same feeder or circuit may have multiple versions within the platform — each representing the state of that feeder at a distinct point in time, or as exported from a different source system (e.g., as-built GIS vs. a planning model scenario). Users can browse, select, and compare any two versions of a feeder from this version history.

The comparison engine operates on this versioned data in a manner conceptually similar to a Git-style code diff. Each network model version is represented internally as a set of structured tabular datasets — one CSV-equivalent table per asset class (e.g., transformers, cable segments, switches, fuses) — where each row describes a single asset with its attributes and connectivity. When two versions are selected for comparison, the engine performs a row-by-row, attribute-by-attribute diff across these datasets, scoped to the feeder level.

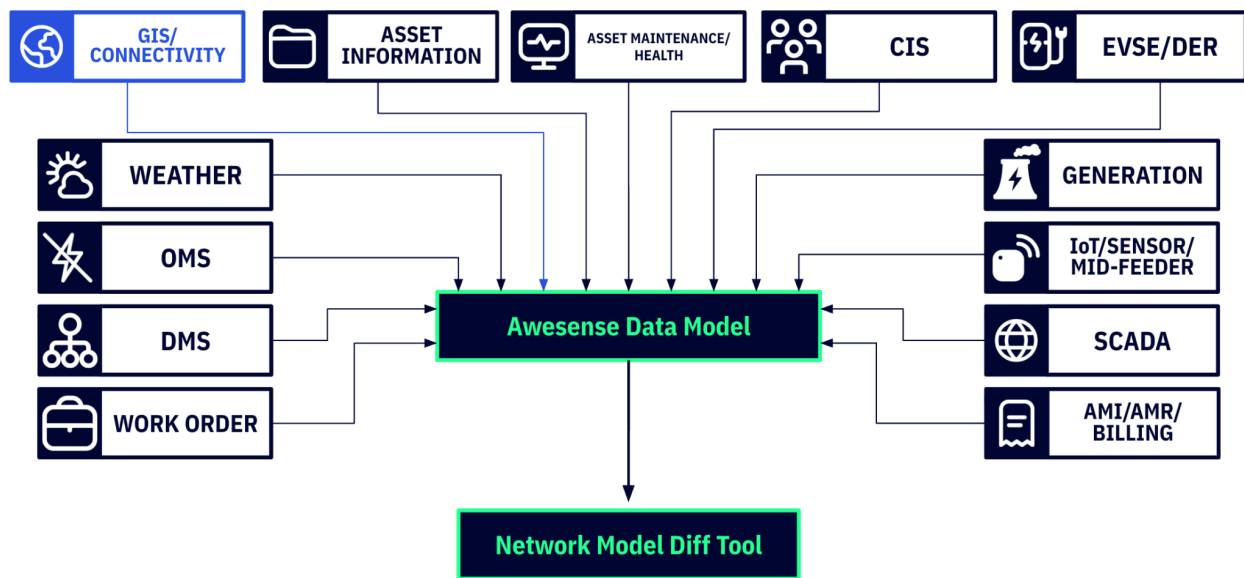
The diff engine classifies every detected difference into one of four change types: additions (rows present in Version B but not Version A), removals (rows present in Version A but absent in Version B), attribute modifications (rows present in both versions but with changed values in one or more columns, such as a rating, phase label, or switch state), and connectivity changes (same assets but wired differently in the two versions). This structured output mirrors the familiar

add/remove/modify paradigm of version control systems, making the results immediately interpretable by engineers accustomed to reviewing code or data diffs.

Diff results are surfaced in a geospatial map view that overlays the feeder on a geographic map with color-coded highlights — green for additions, red for removals, yellow for modifications — so users can immediately see where on the network the changes are concentrated. A structured diff table summarizing all changes is also available for export, supporting change management documentation and model governance workflows.

Required Data Sets

The required data for this use case are visualized in the following diagram. Data sources highlighted in [royal blue](#) are required for this use case.



Available Use Case Implementations:

Users of the Awesense Platform can access this use case by the following means:

1. Please contact [Awesense](#) if you are interested in the implementation of this use case.